Playlist Classifier: Assigning Songs to Playlist Classes

UE24CS352A Machine Learning Mini-Project · Problem 26 · Section E
Team: Nehaa Joshi (PES1UG24CS297) and Nidhi L (PES1UG24CS298)

1. Problem and Objective

This project investigates whether a song can be assigned to a class using numerical audio features. It is inspired by *Training a Playlist Curator Based on User Taste* by Amel Awadelkarim and Kevin Coelho. The available dataset supplies genre labels rather than actual user-created playlist labels, so track genre is used as a proxy for playlist class. The implemented system is a supervised multi-class genre classifier, not a complete personalized playlist recommendation system.

2. Dataset and Preprocessing

The pipeline uses ten numerical audio features: danceability, energy, key, loudness, mode, speechiness, acousticness, liveness, valence and tempo. Twelve classes are selected: ambient, blues, classical, country, gospel, heavy-metal, hip-hop, jazz, opera, punk, reggae and techno.

Preprocessing removes track IDs associated with conflicting labels, filters the selected genres, drops rows with missing feature values and samples up to 150 tracks per class using random seed 42. The resulting dataset contains 1,737 samples; reggae contributes 87 samples and each other class contributes 150. A stratified 80:20 train/test split with seed 42 gives 1,389 training samples and 348 test samples. StandardScaler is fitted on the training split and then applied to the test split.

3. Models and Implementation

Classical baselines are a Perceptron and SVMs with RBF, sigmoid and polynomial kernels. The RBF SVM is tuned over C = {0.1, 1, 10, 100} and gamma = {scale, 0.01, 0.1, 1} using five-fold cross-validation on the training split. The neural network is a feed-forward model with one hidden layer, sigmoid activation, LogSoftmax output, negative log-likelihood loss, Adam optimizer and weight decay 0.0001. Its reported configuration uses 64 hidden neurons, learning rate 0.01 and 300 epochs.

The repository includes preprocessing, training and evaluation scripts, evaluation reports and confusion matrices, and a demo that predicts a genre for a random or user-specified song feature vector.

4. Results

Model	Accuracy	Macro F1	Weighted F1
Perceptron	34.20%	0.2951	0.3053
RBF SVM	48.28%	0.4616	0.4674
Sigmoid SVM	42.82%	0.4099	0.4177
Polynomial SVM	48.56%	0.4818	0.4817
Tuned RBF SVM	48.28%	0.4738	0.4719
Neural network (64)	53.20%	0.51	0.52

The neural network achieved the highest reported test accuracy (53.20%), 4.64 percentage points above the polynomial SVM. The polynomial SVM was the strongest classical baseline by accuracy and macro F1. Neural-network per-class F1 was about 0.71 for heavy-metal, hip-hop and techno, while blues was lower at 0.21.

5. Neural-Network Experiment and Interpretation

Hidden neurons	Test accuracy
12	48.6%
32	50.0%
64	53.2%
128	48.3%

The network reached 65.0% training accuracy and 53.2% test accuracy, an 11.8 percentage-point gap that may indicate overfitting. Test accuracy was used to compare hidden-layer sizes, so the test set was not fully untouched during model selection. The reported neural-network score should therefore be interpreted as a model-selection result rather than a completely independent final estimate. A stricter follow-up would select the hidden size on a validation split and evaluate the test set only once.

Playlist Classifier: Assigning Songs to Playlist Classes

UE24CS352A Machine Learning Mini-Project · Problem 26 · Section E

6. Conclusion and Limitations

In this experiment, the small feed-forward neural network outperformed the tested classical baselines on the selected genre-classification setup. The project demonstrates a reproducible workflow for audio-feature preprocessing, model comparison and prediction. Results are specific to this dataset, feature set, twelve classes and fixed split.

- Genre labels are only a proxy for playlist classes; the system does not model individual taste, playlist context, sequencing or user feedback.
- The reference paper's reported score is not directly comparable because its data and task differ.
- The test set was used to compare neural-network hidden-layer sizes; a validation split and one final test evaluation would give a less biased estimate.
- For more rigorous SVM tuning, scaling should be fitted separately within each cross-validation fold using a pipeline.

7. Team Contributions

Nehaa Joshi (PES1UG24CS297)

- Implemented and experimented with the PyTorch neural network, including different hidden-layer configurations.
- Recorded neural-network experiments, configurations and final evaluation metrics.
- Developed the runnable prediction demo for genre prediction on a sample track.
- Implemented the classical-model held-out evaluation workflow and generated baseline test results, classification reports and confusion matrices.
- Consolidated model comparisons and documented the experimental results, methodology and limitations in the README.

Nidhi L (PES1UG24CS298)

- Created the initial project files and established the base code structure that the team subsequently built upon.
- Created and organised the GitHub repository and committed the initial project files.
- Planned the project workflow and coordinated the division of responsibilities between team members.
- Configured repository access for the team, faculty and teaching assistants, and coordinated with faculty regarding review and demonstration requirements.
- Reviewed the dataset description, preprocessing pipeline, model descriptions and evaluation results documented in the README.
- Prepared the demonstration walkthrough and viva preparation material for the final presentation.

8. Reproducibility

The project uses Python with pandas, NumPy, scikit-learn, PyTorch, matplotlib and joblib. The shared loader uses random seed 42 and fits scaling on the training split. The baseline evaluation script is `src/evaluate_baselines.py`; neural-network training and demo scripts are in `src/`. The local dataset CSV and generated model artifacts are excluded from Git, so the repository README provides dataset acquisition and setup instructions for evaluators.